



Introduction to ReactJS

CSIT340 – Industry Elective 1
Mr. Lance Vincent A. Salera

The Browser

- A **browser** is a program that requests a document from a server and displays it on screen.
 - The **server** returns the document, and the **browser** is responsible for everything that happens to it afterward.
- A browser contains a **rendering engine**, which parses **HTML** and **CSS**, builds the page structure, and paints pixels.

The Browser

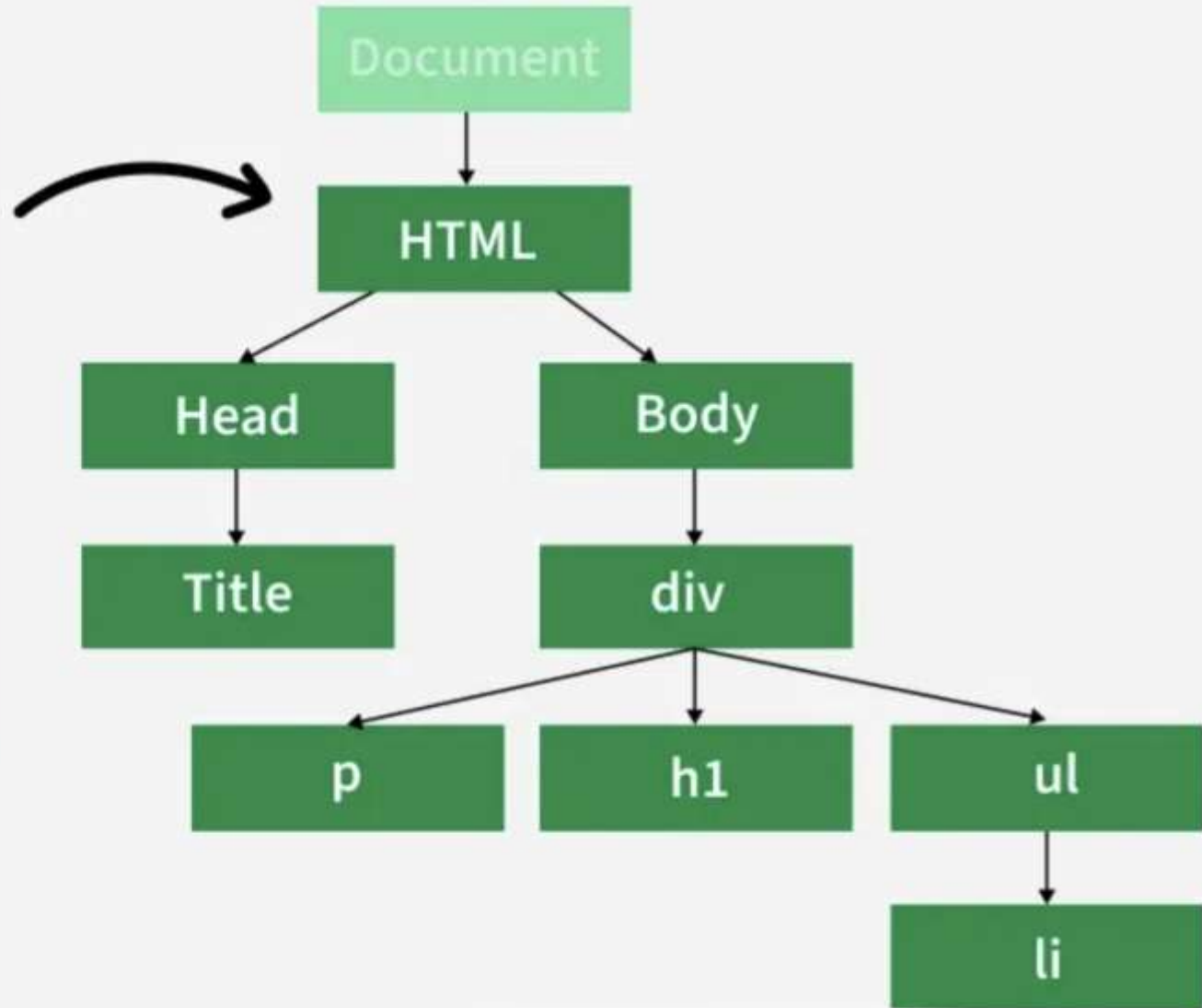
- A browser also contains a **JavaScript engine**, which executes **JavaScript** and paints nothing.
- A page containing no JavaScript displays correctly, because the **rendering engine** alone is sufficient to draw a document.
- *The rendering engine **does not paint** from the **HTML text directly**; it first converts that text into **something else**.*

The Document Object Model

- The **rendering engine** parses the received HTML once and converts it into a **tree of objects** held in memory.
- Each **tag** becomes an **object** in that tree, and nesting in the source becomes a **parent-child relationship** between objects.
- This tree is called the **Document Object Model**, abbreviated **DOM**.

The “DOM Tree”

```
<html>
  <head>
    <title> Webpage Title</title>
  </head>
  <body>
    <div>
      <h1>This is Heading Tag</h1>
      <p>Some Text Content</p>
      <ul>
        <li> List Item</li>
      </ul>
    </div>
  </body>
</html>
```



The Document Object Model

- The pixels on screen are **painted** from the **current state of the tree**
- The HTML text is no longer consulted once parsing is complete.
- The **tree** is built from whatever document the server sent, and the browser has no part in deciding what that document contains.

Web 1.0

- The page was assembled in full on the **server** before it reached the **browser**.
- Early pages were files **stored on disk**, and every visitor received the **identical file**.
- Servers later ran programs that generated pages per request, which changed what was sent without changing when it was assembled.

Web 1.0

- The browser's **only role** in both cases was to **display a finished document** it had been given.
- *Every change to the screen therefore required a different document.*

Web 1.0: reloading

- Following a link or submitting a form causes the browser to **discard the current page** entirely and **request a new document**.
- Parts of the page identical to the ones arriving such as **navigation, headers, and styling** are **retransmitted** and **reparsed** anyway.

Web 1.0: the problem

- **Text typed** into a form is **lost**, because the nodes holding it were destroyed.
- Scroll position resets, and the screen is blank until the new document arrives.
- Replacing the whole page is correct behavior for **reading a document**, where "partially replace the document" has no meaning.

Web 1.0: the problem

- ***Reading*** is not what people eventually wanted to do in a browser.

Web 1.0: the fix

- What the people needed is a **mechanism** where changing one node **does not affect** other nodes in the DOM
- Under the reload model, **changing one value** costs an entire document.
- Applications moved into the browser because every machine already had one, not because it was designed to run programs.

Web 1.0: the fix

- The browser **could not be redesigned**, because the entire internet already depended on its existing behavior.
- Every capability added afterward was built **as a layer** on top of the original design.
- The first such layer used the **engine** that had so far done nothing.

JavaScript

- **JavaScript** runs in the browser's second engine and is given access to the **DOM** through the **global document object**.
- Modifying an object in the tree causes the browser to **repaint** that region **without requesting** a new document.
- The modification exists only in browser memory, and the file on the server is unchanged.

JavaScript

- **Refreshing** discards the modified tree and rebuilds it from the original file.
- A page that **changes after it arrives** is a **different** kind of thing from a **page assembled before it arrives**, and the difference was given a name.

Web 2.0

- Pages could accept input, respond to it, and exchange data with the server **while remaining loaded.**
- Sites built this way carried content produced by their users rather than only by their publishers.

Web 2.0

- No protocol and no language was replaced to make this possible
- The capability came entirely from scripting the DOM.
- **Web 1.0** and **Web 2.0** are labels applied afterward to describe how sites were used, and neither is a version of any technology.



Vanilla JavaScript

Vanilla JavaScript

- **Vanilla JavaScript** refers to the language and the browser's built-in capabilities, with no library or framework added.
- Everything React does is built on top of what this section covers, and nothing in React replaces it.

Vanilla JavaScript: document object

- The **document** used to reach the DOM is not a special keyword
- It is an ordinary **JavaScript object** the browser makes available to every page.
- Understanding what an **object** is explains what **document** is.

Vanilla JavaScript: objects

- An **object** is written as braces containing **key-value pairs** known as **properties**, separated by commas.
- Creating one requires **no class**, **no constructor**, and **no separate file**.
- A property is read by writing the **object's name**, a **dot**, and the **property's key**.

```
const student = { name: "Ana", year: 3 };  
student.name;
```

Vanilla JavaScript: objects

- ***document*** is an object of this kind, and **document.title** reads one of its **properties** in exactly the same way.
- **Every node** in the **DOM** is also an **object**, which is why **textContent** is written as a **property** and assigned to.

Vanilla JavaScript: values and names

- ``const`` names a value that **will not be reassigned** and ``let`` names a value that will be.
- Anything can be named this way -- a **number**, a **string**, an **object**, or a **function**.

```
const name = "Ana";  
let count = 0;
```

Vanilla JavaScript: functions

- A **function** in JavaScript is a **value**, and can be stored, passed to other code, and returned like any other value.
- A function can be written in two forms: a **declaration** and an **expression**.

Vanilla JavaScript: functions

- A **function declaration** begins with the **function** keyword and gives the function a name directly, without an assignment.
- The body is enclosed in **braces** and must use **return** to produce a value.

```
function double(n) {  
  return n * 2;  
}
```

Vanilla JavaScript: functions

- A **function expression** produces a function as a value and assigns it to a name with **const**

```
const double = function(n) {  
  return n * 2;  
}
```

```
const double = (n) => {  
  return n * 2;  
}
```

```
const double = (n) => n * 2;
```

Vanilla JavaScript: functions

- An **arrow function** is a function expression written as a **parameter list**, the **token =>**, and the **function's body**.
- A **braced body** contains statements and must use **return** to produce a value.
- A **body** written as a single expression returns that expression, with no braces and no **return** keyword.

Vanilla JavaScript: functions

- **Declarations** and **expressions** produce functions that behave identically when called, and both appear throughout published React code.

Vanilla JavaScript: function calling

- A function's name written alone evaluates to the function.
- A function's name followed by parentheses runs the function immediately and evaluates to whatever it returned.

```
const double = (n) => n * 2;
```

```
double // returns "() => n * 2"
```

```
double(5) // returns 10
```

Vanilla JavaScript: function calling

- Passing **double** hands over something that can be **run later**, while passing **double()** hands over a result and **runs nothing later**.
- Which form is correct depends entirely on whether the receiving code intends to call it.

Events

- An **event** is a notification the browser produces when something happens.
- Every event is associated with the node it happened to.
- **Clicking, typing, submitting a form, and finishing a page load** each produce a different event.

Events

- A program **cannot predict** when an event will occur.
- Code that responds to an event must therefore be **registered** before the event happens.

Event Handlers

- A **function** registered to run when an event occurs is called an **event handler**.
- Assigning a function to a node's **onclick** property registers that function as the handler for clicks on that element.

```
document.getElementById("add").onclick = add;
```

Event Handlers

- Assigning a function to a node's **onclick** property registers it to run on every click of that node.
- To register the function properly, it should be written **without parentheses**, i.e., a **function reference**
- A **function call** (with parentheses) calls the function once at the moment the line runs and stores its return value instead.

Event Handlers

- A handler property holds **one function**, and assigning a second one replaces the first.

```
document.getElementById("add").onclick = add; // registered  
document.getElementById("add").onclick = add() // called once, nothing registered
```

Event Handlers: `addEventListener`

- `addEventListener("click", add)` registers a handler and **appends** it to any handlers already registered on that node.
- The event name passed to the method has no **on** prefix, unlike the property name.

```
// both run on every click  
button.addEventListener("click", add);  
button.addEventListener("click", log);
```

<script>

- A **script** element is where **JavaScript** is placed in an **HTML document**.
- A **src** attribute loads the **JavaScript** from a **separate file** instead of the element's contents.
- Code in a script element runs as part of parsing the document.

```
<script>  
  console.log("runs here");  
</script>
```

```
<script src="app.js"></script>
```

<script>: timing

- The **browser** reads an HTML document from **top to bottom** and builds the tree as it goes.
- Reaching a script element **stops** that process: *the code runs to completion before parsing continues*.
- Elements **above** the script exist in the tree when the code runs, and elements **below** it do not.

<script>: timing

- **getElementById** returns **null** for an element that does not yet exist, and reading a property of null raises an error.
- Placing a script **after every element** in the document guarantees the whole tree exists before the code runs.

<script>: timing

- A **script** in the **head** runs before any element exists and is safe only if it touches no elements.
- Code registered inside a **DOMContentLoaded** handler runs after the entire tree is built, regardless of where the script sits.

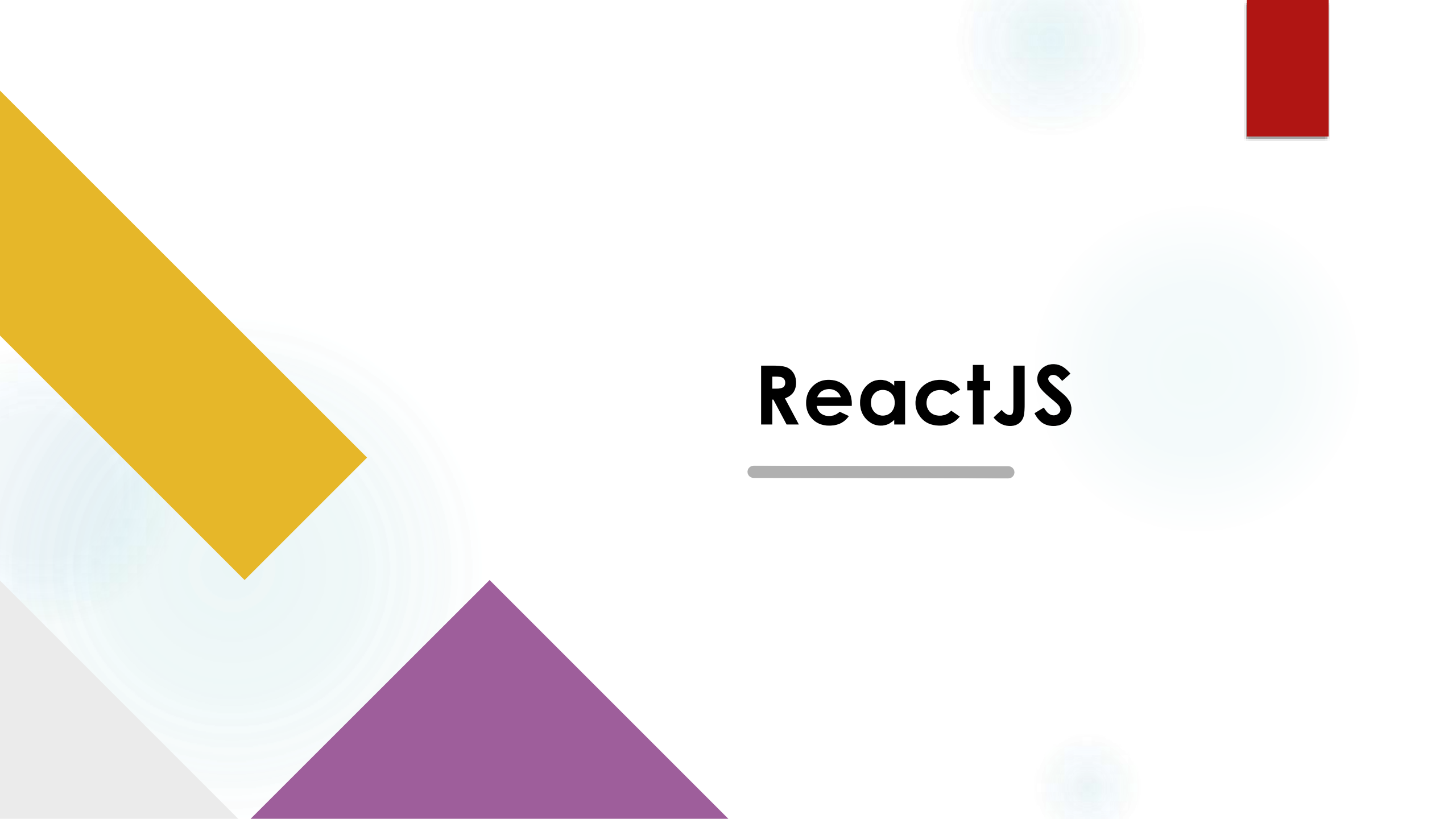
<script>

- The **defer** attribute delays execution until parsing is complete, from any position in the document.
- **type="module"** applies that deferral automatically, so a module script never runs too early.

```
<script defer src="app.js"></script>  
<script type="module" src="app.js"></script>
```

<script>

- What matters is **when the code runs**, not where the tag is written.
- Content written **after the closing body** tag is still placed inside body by the parser, so position in the source is not a reliable rule.



ReactJS

VanillaJS: limitations

- Changing a variable **does not change** the screen.
- Only a line that touches a node changes the screen.

```
count += 1; // screen unchanged  
display.textContent = count; // screen changed
```

VanillaJS: limitations

- The browser runs your code on its own only for **events** it can observe, such as a **click** or a **keypress**.
- A variable changing is **not an event**, so there is nothing to register a handler for.

VanillaJS: limitations

- *The screen has to be updated at **every place** the **data changes**: every handler, every timer, every server response.*

ReactJS

- **React** is a **JavaScript library** that produces a page from data
- The developer writes **what the page contains** for a given set of data and writes no instruction to locate or modify a node.
- React produces the page again whenever the data changes, and updates only the parts of the browser's page that differ.
- The developer does not call it.

ReactJS

- **Text typed** into inputs, **keyboard focus**, and **scroll position** survive an update, because the nodes holding them are **not replaced**.

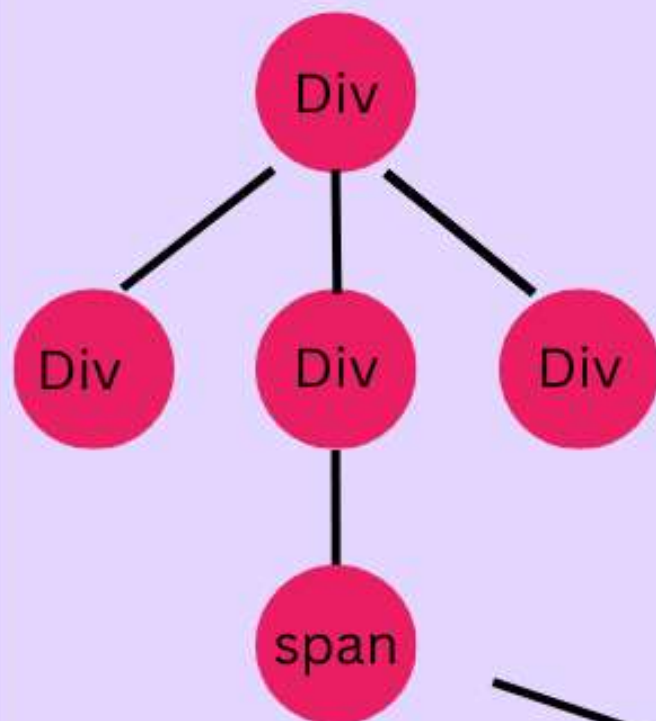
ReactJS: updating the DOM

- The browser still paints from the DOM tree and **React** does not replace it.
- React keeps its own **tree of objects** describing what the page should contain.
- When the data changes, React produces a **new version** of its own tree and compares it against the previously rendered one.

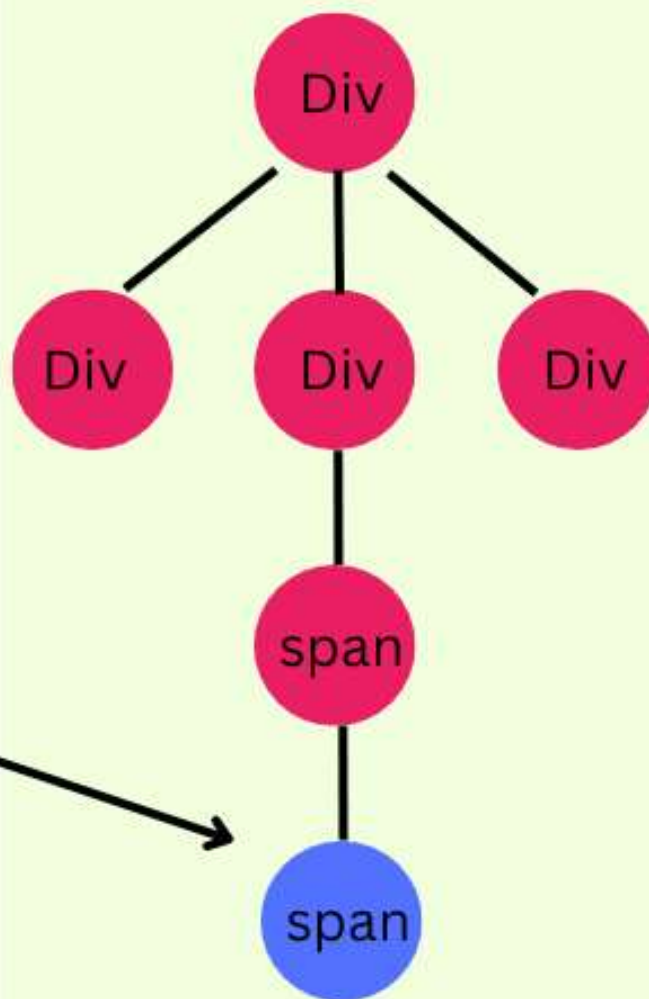
ReactJS: updating the DOM

- The comparison is between **React's** own **two versions**, and the DOM is never read during it.
- React's tree is the **source of truth**, and the DOM is what is produced from it.

Existing Virtual DOM



After Update



Render to Original DOM

